NLP Lab Report: Trigrams and Word Embeddings

Natural Language Processing / Computational Linguistics
Course Assignment — Jurafsky & Martin, Ch. 2, 3, & 6

Malini Senthilkumar (PhD Biomedical Data Science)

Abstract

This report presents the results of a Natural Language Processing lab assignment exploring statistical and neural approaches to language modeling. Using the NLTK webtext wine review corpus (wine.txt), we preprocessed 1,230 sentences, constructed a trigram language model with linear interpolation, evaluated it via perplexity on a held-out test set, and trained a Word2Vec Skip-gram model to learn distributional word representations. We report trigram probabilities, test-set perplexity, nearest-neighbor results, and a comparative analysis of the two approaches.

1. Introduction

N-gram language models and word embeddings represent two foundational paradigms in NLP. N-gram models estimate the probability of a word given its preceding context using co-occurrence counts, while word embeddings such as Word2Vec learn dense vector representations that capture distributional semantics. This lab implements both approaches on a wine review corpus and compares their strengths and limitations.

2. Data and Preprocessing

The dataset is the wine.txt file from the NLTK webtext corpus. Preprocessing involved splitting the raw text into lines, converting to lowercase, removing all punctuation using the regular expression pattern r'\w+', and discarding empty lines. This produced 1,230 tokenized sentences. The corpus was then split into 80% training (984 sentences) and 20% test (246 sentences) partitions.

A vocabulary was constructed from the training set using a minimum frequency threshold of 3. Words occurring fewer than 3 times were replaced with the <UNK> token. The resulting vocabulary contained 897 types including the special tokens <s>, </s>, and <UNK>. Each sentence was padded with two start tokens (<s> <s>) and one end token (</s>) for trigram extraction.

3. Trigram Language Model

3.1 N-Gram Counts

Unigram, bigram, and trigram counts were extracted from the padded training sentences. The training set yielded 897 unique unigrams, 9,536 unique bigrams, and 16,528 unique trigrams over a total of 22,782 tokens.

3.2 Trigram Probability with Linear Interpolation

The maximum likelihood estimate (MLE) of a trigram probability is defined as P(w3 | w1, w2) = C(w1, w2, w3) / C(w1, w2). To address data sparsity, we applied linear interpolation combining trigram, bigram, and unigram estimates:

$$P_interp = 0.6\ P(w3 \mid w1, w2) + 0.3\ P(w3 \mid w2) + 0.1\ P(w3)$$

To compute P(tannins | firm, ripe), the individual components are:

- P_trigram(tannins | firm, ripe) = 0.000000
- P_bigram(tannins | ripe) = 0.085714
- P_unigram(tannins) = 0.001492

The interpolated probability is 0.6(0.000000) + 0.3(0.085714) + 0.1(0.001492) = 0.025864. The trigram "firm ripe tannins" was never observed in training, yielding a zero MLE trigram estimate. Interpolation recovers a nonzero probability primarily through the bigram component, where "ripe" is followed by "tannins" approximately 8.6% of the time.

4. Perplexity Evaluation

Perplexity was computed on the 246 test sentences (4,860 tokens) using the interpolated trigram model. The formula used was:

$$PP = 2 \wedge (-1/N * sum(log2\ P(wi \mid wi\text{-}2, wi\text{-}1)))$$

A probability floor of 1e-10 was applied to prevent undefined logarithms for any zero-probability events. The resulting test-set perplexity was 144.64. This indicates that on average the model is as uncertain as choosing uniformly among approximately 145 candidate words at each position, which is reasonable for a small domain-specific corpus.

5. Word2Vec Skip-Gram Model

A Word2Vec Skip-gram model was trained on the training sentences with sentence-boundary markers removed. The model was configured with vector_size=100, window=5, sg=1 (Skip-gram), min_count=2, and 50 training epochs. The resulting model vocabulary contained 895 words.

The five nearest neighbors of "tannins" by cosine similarity were:

Rank	Word	Cosine Similarity
1	succulent	0.5758
2	classy	0.5138
3	gives	0.5050
4	evident	0.4977
5	softer	0.4888

These neighbors reflect wine-tasting vocabulary associated with texture and mouthfeel. Words describing structural wine qualities appear in similar distributional contexts as "tannins" and are therefore located nearby in the embedding space.

6. Analysis

6.1 Why can a trigram model assign probability zero to a reasonable phrase that was not observed during training?

A trigram model estimates P(w3 | w1, w2) directly from corpus counts: C(w1, w2, w3) / C(w1, w2). If the specific three-word sequence was never observed in the training data, the numerator is zero and the resulting probability is exactly zero, regardless of whether the phrase is linguistically plausible. Because the space of possible trigrams grows as V cubed, even a large training corpus will leave the vast majority of valid trigrams unobserved. This is the fundamental sparsity problem of count-based n-gram models.

6.2 Why does taking log2(0) cause a problem when calculating perplexity?

The logarithm of zero is mathematically undefined; log2(0) tends to negative infinity. In the perplexity formula, we compute the sum of log2 P(wi | context) across all test tokens. If any single token receives a probability of exactly zero, that term becomes negative infinity, which dominates the entire sum and produces an infinite perplexity. In practice, this means that one unseen trigram can render the model evaluation meaningless, even if the model predicts every other word accurately. Smoothing methods or a probability floor are therefore essential to ensure that no event receives exactly zero probability.

6.3 How can Word2Vec identify words related to "tannins" even when a particular word sequence never appeared in the training corpus?

Word2Vec learns dense vector representations by optimizing over local context windows rather than memorizing specific n-gram sequences. During training, the Skip-gram objective pushes words that share similar surrounding contexts toward nearby positions in the vector space. For instance, "tannins" and "succulent" may both frequently co-occur with descriptors such as "firm," "dry," "ripe," and "oak," even if they never appear adjacent to each other. This distributional hypothesis, often paraphrased as "you shall know a word by the company it keeps" (Firth, 1957), enables Word2Vec to capture semantic relationships that count-based n-gram models, which require exact sequence matches, fundamentally cannot represent.

7. Conclusion

This lab demonstrated two complementary approaches to modeling language. The trigram model with linear interpolation achieved a test-set perplexity of 144.64 and successfully assigned nonzero probability to unseen sequences through interpolation. The Word2Vec Skip-gram model captured meaningful semantic relationships, identifying wine-tasting terms related to "tannins" purely from distributional context. While n-gram models answer "what word comes next," word embeddings answer "which words mean similar things." Both remain foundational tools in modern NLP.

References

Firth, J. R. (1957). A synopsis of linguistic theory, 1930-1955. Studies in Linguistic Analysis.

Jurafsky, D., & Martin, J. H. (2024). Speech and Language Processing (3rd ed.). Chapters 2, 3, & 6.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.